

CSE331

AUTOMATA AND COMPUTABILITY

Regular Expression

Summer 2026

Today's learning goals

Sipser Ch 1.3

- Decide whether or not a string is described by a given regular expression
- Design a regular expression to describe a given language

Recap

Regular sets of strings i.e., those that aren't too complicated are those that are

- the language of some DFA, or equivalently,
- the language of some NFA.

If a set of strings can be expressed as the result of **complement, union, intersection, concatenation, Kleene star** of regular languages, then it itself is regular.

Regular expressions

- Can all regular languages be expressed as those that are built up from **very simple** languages using the regular operations?

DFAs vs Regular Expressions

- DFAs – **check** if a given word is in the language
- Regular Expressions – **generate** all words in the language

The regular operations

Sipser Def 1.23 p. 44

For A, B languages over same alphabet, define:

$$A \cup B = \{x \mid x \in A \text{ or } x \in B\}$$

$$A \circ B = \{xy \mid x \in A \text{ and } y \in B\}$$

$$A^* = \{x_1x_2 \dots x_k \mid k \geq 0 \text{ and each } x_i \in A\}$$



These are operations on sets!

Inductive application of closure Sipser 1.52 p. 64

R is a **regular expression** over Σ if

1. $R = a$, where $a \in \Sigma$
2. $R = \varepsilon$
3. $R = \emptyset$
4. $R = (R_1 \cup R_2)$, where R_1, R_2 are themselves regular expressions
5. $R = (R_1 \circ R_2)$, where R_1, R_2 are themselves regular expressions
6. (R_1^*) , where R_1 is a regular expression.

Watch out for
overloaded symbols!

Σ is shorthand for $(0 \cup 1)$ if $\Sigma = \{0, 1\}$, Parentheses may be omitted, R^+ means RR^* , R^k means R concatenated with itself k times

Syntax $\square$ Languages

The language described by a regular expression, $L(R)$:

- $L((0 \cup 1) \cup 1) = \{ \quad \quad \quad \}$
- $L(1^* 1) = \{ \quad \quad \quad \}$
- $L(1^* \emptyset 0) = \{ \quad \quad \quad \}$

1. $R = a$, where $a \in \Sigma$
2. $R = \varepsilon$
3. $R = \emptyset$
4. $R = (R_1 \cup R_2)$
5. $R = (R_1 \circ R_2)$
6. (R_1^*)

Syntax □ Languages

The language described by a regular expression, $L(R)$:

- $L(\Sigma^*(00+11)\Sigma^*) = \{ \quad \}$
- $L(\Sigma(\Sigma\Sigma)^*) = \{ \quad \}$
- $L(0^*10^*10^*) = \{ \quad \}$

1. $R = a$, where $a \in \Sigma$
2. $R = \varepsilon$
3. $R = \emptyset$
4. $R = (R_1 \cup R_2)$
5. $R = (R_1 \circ R_2)$
6. (R_1^*)

Syntax □ Languages

The language described by a regular expression, $L(R)$:

- $L((0U1) \circ (0U1) \circ (0U1))^* = \{$ $\}$

1. $R = a$, where $a \in \Sigma$
2. $R = \varepsilon$
3. $R = \emptyset$
4. $R = (R_1 \cup R_2)$
5. $R = (R_1 \circ R_2)$
6. (R_1^*)

L(R)

Which of the following strings is **not** in the language described by

$$(((00)^*(11)) \cup 01)^*$$

- A. 00
- B. 01
- C. 1101
- D. ε
- E. I don't know

L(R)

Let L be the language over $\{a,b\}$ described by the regular expression

$$((a \cup \emptyset) b^*)^*$$

Which of the following is **not** true about L?

- A. Some strings in L have equal numbers of a's and b's
- B. L contains the string aaaaaa
- C. a's never follow b's in any string in L
- D. L can also be represented by the regular expression $(ab^*)^*$
- E. More than one of the above.

L(R)

Review: Determine whether each statement below about regular expressions over the alphabet $\{a, b, c\}$ is true or false:

True or False: $ab \in L((a \cup b)^*)$

True or False: $ba \in L(a^*b^*)$

True or False: $ab \in L(a \cup b \cup c)$

True or False: $\varepsilon \in L((a \cup b)^*)$

True or False: $\varepsilon \in L(aa^* \cup bb^*)$

Writing Regular Expression



Example

Write regular expression which generates

$$L_1 = \{w \mid \text{length of } w \text{ is } 1 \text{ or more}\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_2 = \{w \mid w \text{ contains at least two 1s}\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_3 = \{w \mid w \text{ contains at most two } 1s\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_4 = \{w \mid \text{length of } w \text{ is not divisible by } 3\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_5 = \{w \mid 0s \text{ and } 1s \text{ alternate in } w\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_6 = \{w \mid w \text{ doesn't contain } 00 \text{ and } 11\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_7 = \{w \mid w \text{ doesn't end with } 01\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_8 = \{w \mid w \text{ doesn't contain } 00\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_9 = \{w \mid w \text{ doesn't contain } 10\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_{10} = \{w \mid w \text{ starts and ends with different symbols}\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_{11} = \{w \mid w \text{ starts and ends with the same symbols}\}$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_{12} = \{w \mid w \text{ has an even number of copies of the substring } ab\}$$

The main observation needed here is that the way to get two copies of **ab** is to have a nonempty run of a's, then a nonempty run of b's, then a nonempty run of a's, and then another nonempty run of b's. To get an even number of copies of **ab**, you need to repeat this pattern multiple times. Finally, accounting for what comes before that first run of a's and after that last run of b's, we can come up with this regex:

$$b^*(a^+b^+a^+b^+)^*a^*$$

Writing Regular Expression



Example

Write regular expression which generates

$$L_{13} = \{w \mid w \neq ab\}$$

This one is a little trickier than it looks because there is no easy way to say “everything but **ab**” in regex-speak. We need to figure out what structure the other strings all share that **ab** doesn’t.

There are a lot of ways to solve this one, but my personal favorite is to use the following observation. Any string that isn't **ab** is either

- length 0, 1, or 3 or more, or
- length 2 and not **ab**.

We can directly encode option (1) and enumerate all choices for option (2) to get this regex:

$$\underbrace{\Sigma^? \cup \Sigma^2\Sigma^+}_{\text{length 0, 1, or 3 or more}} \cup \underbrace{aa \cup b\Sigma}_{\text{length 2 and not ab}}$$

Writing Regular Expression



Example

$L_{14} = \{ w \mid \text{number of 0s in } w \text{ is a multiple of } 3 \}$

$L_{15} = \{ w \mid \text{every 1 is followed by at least one 0} \}$

$L_{16} = \{ 1^m 0^n \mid m, n \geq 1 \}$

$L_{17} = \{ L_{14} \cap L_{15} \cap L_{14} \}$



Regular expressions in practice

- **Compilers:** first phase of compiling transforms Strings to Tokens **keywords, operators, identifiers, literals**
 - One regular expression for each token type
- **Other software tools:** grep, Perl, Python, Java, Ruby, ...

"Regular = regular"

Sipser Theorem 1.54 p. 66

Theorem: A language is regular if and only if some regular expression describes it.

Lemma 1.55: If a language is described by a regular expression, then it is regular.

Lemma 1.60: If a language is regular, then it is described by some regular expression